

Exposé: Sicherheitsaspekte von WebAssembly – Analyse und praktische Untersuchung von Sicherheitslücken

1. Problemstellung und Relevanz

WebAssembly (Wasm) hat sich in den letzten Jahren als leistungsfähige Technologie zur Ausführung von Code im Webbrowser etabliert. Durch die Möglichkeit, Programme in Sprachen wie C, C++ oder Rust zu kompilieren und im Browser auszuführen, eröffnet WebAssembly neue Potenziale hinsichtlich Performance und Plattformunabhängigkeit. Gleichzeitig entstehen jedoch neue Herausforderungen im Bereich der IT-Sicherheit.

Obwohl WebAssembly mit einem sicherheitsorientierten Design entwickelt wurde (z. B. durch Sandboxing und kontrollierten Speicherzugriff), ist bislang nicht abschließend geklärt, inwieweit diese Mechanismen in der Praxis ausreichend sind oder ob neue Angriffsvektoren entstehen. Diese Arbeit untersucht daher systematisch die Sicherheitsaspekte von WebAssembly und analysiert potenzielle Schwachstellen anhand konkreter Beispiele.

2. Zielsetzung der Arbeit

Ziel der Arbeit ist es, die Sicherheitsmechanismen von WebAssembly zu analysieren und deren Wirksamkeit kritisch zu bewerten. Darüber hinaus sollen mögliche Sicherheitslücken identifiziert und in einem praktischen Teil demonstriert werden.

Die Arbeit verfolgt dabei folgende konkrete Ziele:

- Analyse der Sicherheitsarchitektur von WebAssembly
 - Identifikation potenzieller Angriffsvektoren
 - Vergleich mit klassischen Web-Sicherheitsmodellen (z. B. JavaScript)
 - Praktische Demonstration ausgewählter Sicherheitslücken anhand von Beispielen
-

3. Forschungsfragen

Die Arbeit orientiert sich an folgenden zentralen Forschungsfragen:

1. Welche Sicherheitsmechanismen implementiert WebAssembly standardmäßig?
2. Inwiefern schützt WebAssembly vor klassischen Webangriffen (z. B. XSS, Speicherfehler)?
3. Welche neuen Angriffsvektoren entstehen durch den Einsatz von WebAssembly?
4. Welche praktischen Sicherheitslücken lassen sich anhand konkreter Implementierungen demonstrieren?
5. Wie kann die Sicherheit von WebAssembly-Anwendungen verbessert werden?

4. Methodik

Die Arbeit kombiniert theoretische Analyse mit praktischer Untersuchung:

Theoretischer Teil:

- Literaturrecherche zu WebAssembly und IT-Sicherheit
- Analyse der Spezifikation und Sicherheitsmechanismen (Sandboxing, Memory Isolation, Validierung)
- Untersuchung bekannter Sicherheitsprobleme und Forschungsergebnisse

Praktischer Teil:

- Entwicklung kleiner WebAssembly-Beispielanwendungen
 - Implementierung potenziell unsicherer Szenarien (z. B. unsicherer Speicherzugriff in C/C++)
 - Analyse und Demonstration von Sicherheitslücken (z. B. Buffer Overflows, Side-Channel-Angriffe)
 - Testen mit geeigneten Tools (z. B. Browser DevTools, Debugger, Fuzzing-Tools)
-

5. Vorläufige Gliederung

1. Einleitung
 2. Grundlagen
 - WebAssembly: Architektur und Funktionsweise
 - Sicherheitskonzepte im Web
 3. Sicherheitsmechanismen von WebAssembly
 - Sandboxing
 - Speicherverwaltung
 - Validierung und Typsicherheit
 4. Analyse potenzieller Sicherheitsrisiken
 - Klassische Schwachstellen (z. B. Speicherfehler)
 - Neue Angriffsvektoren (z. B. Side Channels, Spectre/Meltdown-Bezug)
 5. Praktischer Teil: Demonstration von Sicherheitslücken
 - Aufbau der Testumgebung
 - Beschreibung der Beispiele
 - Analyse der Ergebnisse
 6. Diskussion und Bewertung
 7. Fazit und Ausblick
-

6. Erwartete Ergebnisse

Es wird erwartet, dass WebAssembly zwar durch sein Design ein hohes Sicherheitsniveau bietet, jedoch insbesondere durch die Nutzung unsicherer Programmiersprachen (z. B. C/C++) weiterhin klassische Schwachstellen auftreten können. Zudem könnten neue Risiken durch die Interaktion mit JavaScript und durch mikroarchitektonische Angriffe entstehen.

7. Zeitplan

Abgabe: 15. August 2026(15 Wochen)

- Woche 1–4: Literaturrecherche und Einarbeitung
 - Woche 4–6: Theoretische Analyse
 - Woche 7–10: Praktische Umsetzung und Experimente
 - Woche 11-12: Auswertung und Schreiben
 - Woche 13: Überarbeitung und Finalisierung
 - Woche 14-15: Puffer
-

8. Literatur

- Official Wasm Documentation: <https://webassembly.org/docs/security/>
- W3C WebAssembly Specification
- Diverse aktuelle Forschungspaper zu WebAssembly-Sicherheit
- WebAssembly and security: A review:
<https://www.sciencedirect.com/science/article/abs/pii/S157401372500005X>